

... different distributed DBMS architectures. Apply fragmentation and replication strategies effectively. Analyze CAP/PACELC trade-offs in system design. Choose appropriate consistency models based on application needs. Implement distributed transactions using 2PC, 3PC, Paxos, or Raft.	Unit IV: Distributed Database Systems 4.1 Architectural models for distributed DBMSs - Client/Server systems - Peer-to Peer systems - <u>Multidatabase</u> systems 4.2 Distributed Database Architectures (Fragmentation, Replication) 4.3 CAP Theorem (Consistency, Availability and Partition <u>Tolerance</u>) and PACELC Trade-offs 4.4 Consistency Models (Strong, Eventual, Causal) 4.5 Distributed Transactions (2PC, 3PC, Paxos, Raft) 4.6 Security in Distributed Database (<u>Authentication</u>, Authorization, Encryption, Integrity)	8 hrs
---	---	---------------------

Yuba Raj Devkota

NCIT

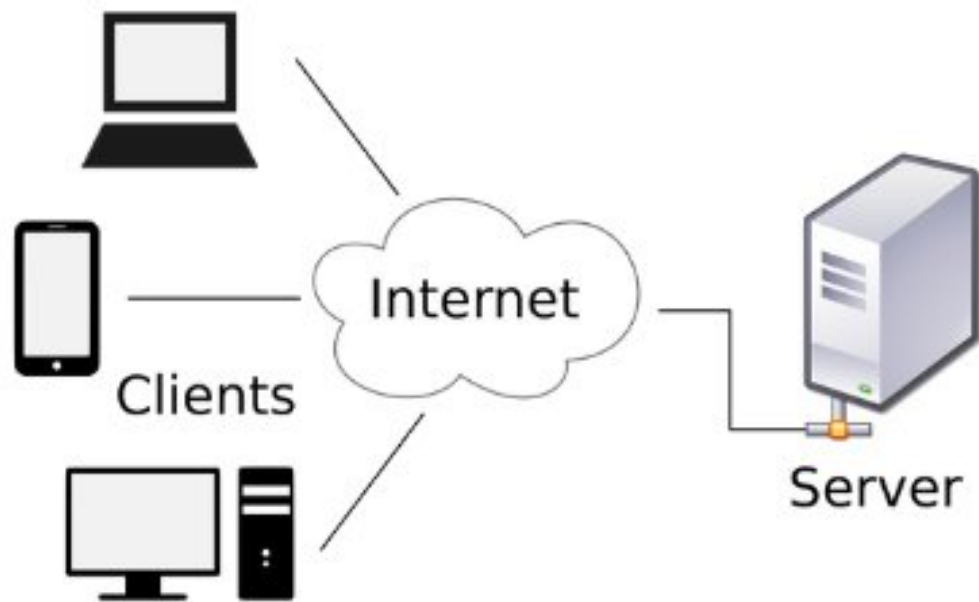
Architectural Models for Distributed DBMS

1. Client/Server Systems

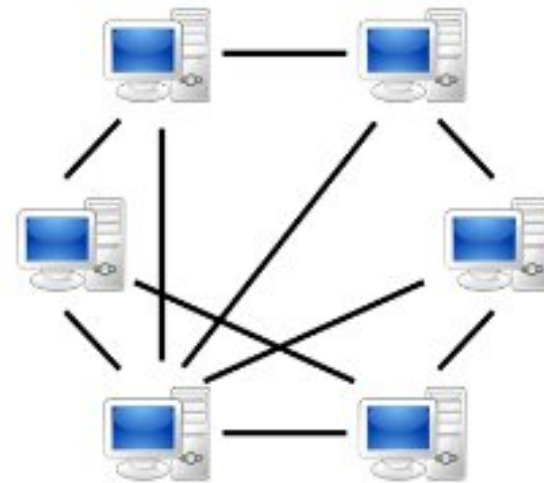
- **Definition:** A centralized server manages the database, while clients request and process data.
- **Roles:**
 - **Server:** Stores data, processes queries, enforces security.
 - **Client:** Requests data, performs local processing.
- **Advantages:**
 - Centralized control.
 - Scalable by adding more clients.
- **Disadvantages:**
 - Server is a single point of failure.
 - Network congestion if too many clients.

Example:

- A **banking system** where the main server holds all account data, and ATM machines (clients) request balance checks.



Client-Server



Peer-to-Peer

2. Peer-to-Peer (P2P) Systems

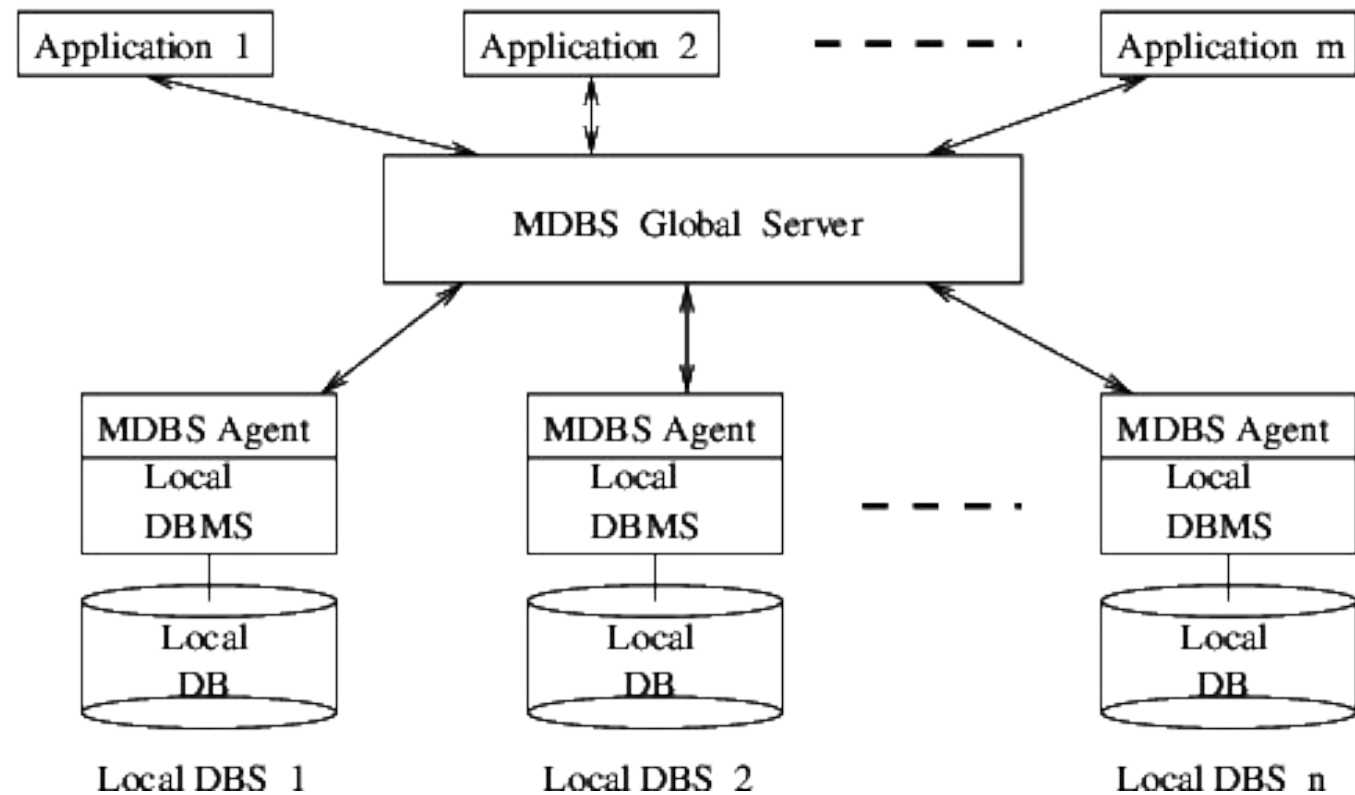
- **Definition:** All nodes (peers) act as both clients and servers, sharing data without a central authority.
- **Advantages:**
 - No single point of failure.
 - Highly scalable.
- **Disadvantages:**
 - Data consistency is hard to maintain.
 - Security risks (unauthorized access).

Example:

- **BitTorrent file sharing**, where each peer stores and shares parts of files.

3. Multidatabase Systems (Federated Databases)

- **Definition:** Integrates multiple independent databases into a unified system.
- **Types:**
 - **Loosely Coupled:** No global schema; users access individual databases.
 - **Tightly Coupled:** A global schema integrates all databases.
- **Advantages:**
 - Allows interoperability between different DBMS.
- **Disadvantages:**
 - Complex query processing.
 - Schema integration challenges.



Example:

- A **university system** combining separate databases for admissions, library, and finance.

What is distributed computing? Discuss various types of DDMS Architectures, focusing on the three key aspects: autonomy, distribution and heterogeneity. [PU, 2025]

What is Distributed Computing?

Distributed computing is a computing model where **multiple computers (nodes)** work together to perform tasks. These nodes communicate via a network and appear as a **single unified system** to the user.

Key idea:

- Tasks are divided and executed across different machines.
- Each machine may have its own processor, memory, and storage.
- Workload is shared → improves **performance, reliability, and scalability**.

Simple example:

- Google Search uses thousands of servers to handle user queries.
- A college website hosted on multiple servers for load balancing.

✔ Distributed Database Management System (DDMS) Architectures

DDMS architecture describes how databases are structured and managed across multiple locations.

When analyzing DDMS, we focus on **three aspects**:

- 1. **Autonomy**
- 2. **Distribution**
- 3. **Heterogeneity**

Below is an easy explanation of each, with types and examples.

Aspect	Types	Meaning	Simple Example	📄
Autonomy	Tight, Loose, High	How independently sites operate	Bank branches (tight), University departments (loose), Hospitals (high)	
Distribution	Centralized, Replicated, Partitioned, Hybrid	How data is stored	Facebook replication, Branch wise partitioning	
Heterogeneity	Hardware, Software, DBMS, Data model	Differences among systems	MySQL + Oracle + MongoDB in different sites	

◆ 1. Autonomy in DDMS

Autonomy refers to how independently each site (node) can operate.

Types of Autonomy

a) Tight Coupling (Low autonomy)

Sites work closely together and share control.

✓ Characteristics:

- Centralized control
- Strong coordination
- Less freedom at local sites

✓ *Example: A bank's core database where all branches strictly follow rules from the head office.*

b) Loose Coupling (Medium autonomy)

Sites have some independence but follow common protocols.

✓ *Example: University departments maintain their own databases, but they are connected to a central system for student information.*

c) High Autonomy

Sites operate fully independently; coordination is minimal.

✓ *Example: Different hospitals maintaining their own patient databases separately, but sharing selective data when required.*

◆ 2. Distribution in DDMS

Distribution explains how data is physically stored across sites.

Types of Distribution

a) Centralized Database

Only one site stores and manages the database.

✓ Example:

- A small company storing all data in one main server.

c) Partitioned (Fragmented) Database

Database is divided into fragments across sites.

✓ Example:

- A country-wide sales company:
 - Kathmandu branch stores Kathmandu customer data
 - Pokhara branch stores Pokhara customer data

This is **horizontal fragmentation**.

b) Replicated Database

Copies of the same data exist on multiple sites.

✓ Advantages:

- Faster access
- Fault tolerance

✓ *Example: Facebook stores user data in multiple data centers worldwide for faster access.*

d) Hybrid (Replication + Fragmentation)

Some fragments are replicated, others are stored separately.

✓ Example:

- Student data split by semesters (partitioning)
- Exam results replicated across all campuses (replication)

◆ 3. Heterogeneity in DDMS

Heterogeneity refers to differences in hardware, software, DBMS, or data models across sites.

Types of Heterogeneity

a) Hardware Heterogeneity

Different machines used at different sites.

✓ *Example: One site uses Windows server, another uses Linux server.*

c) DBMS Heterogeneity

Different database systems.

✓ **Example:**

- One site uses MySQL
- Another uses Oracle
- A third uses PostgreSQL

DDMS must integrate them smoothly.

b) Software Heterogeneity

Different operating systems or applications.

✓ **Example:**

- Kathmandu branch uses Java apps,
- Lalitpur branch uses .NET apps.

d) Data Model Heterogeneity

Data stored in different formats or models.

✓ **Example:**

- One branch uses relational tables
- Another uses NoSQL documents (MongoDB)

Distributed Database Architectures

1. Fragmentation

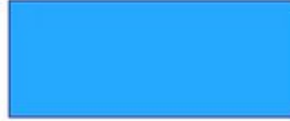
- **Definition:** Splitting a database into smaller parts (fragments) stored at different locations.
- **Types:**
 - **Horizontal Fragmentation:** Rows of a table are distributed.
 - *Example:* A `Customers` table split by region (Asia, Europe, etc.).
 - **Vertical Fragmentation:** Columns of a table are distributed.
 - *Example:* A `Student` table split into `Student_Personal` (name, DOB) and `Student_Academic` (grades, courses).
 - **Mixed (Hybrid) Fragmentation:** Combines both horizontal and vertical.

- The **Employees** table is split into:
 - **Employees_Private** (SSN, salary, address)
 - **Employees_Public** (name, department, email)

```
-- HR needs private + public data (requires a join)
SELECT E.name, EP.salary
FROM Employees_Public E
JOIN Employees_Private EP ON E.emp_id = EP.emp_id;
```

Fragmentation

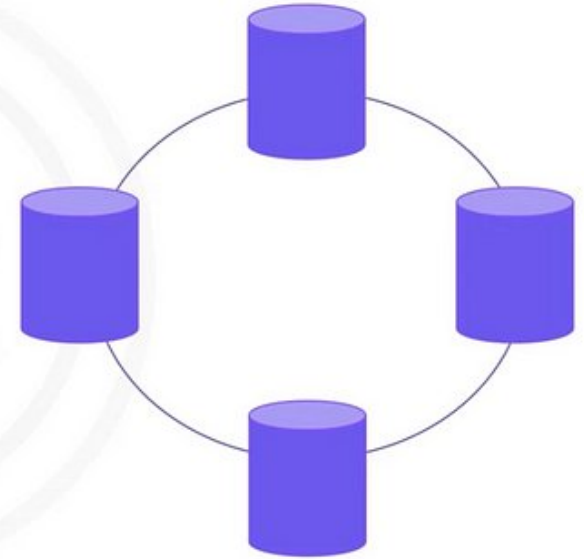
Input data



Fragmented input data



- Fragmenting your data (partitioning, sharding)



Advantages:

- Improved query performance (local access).
- Better security (sensitive data stored separately).

Disadvantages:

- Complex joins across fragments.
- Recovery is harder if a fragment is lost.

2. Replication

- **Definition:** Storing multiple copies of data at different sites.
- **Types:**
 - **Full Replication:** Entire database is copied to all sites.
 - *Example:* A global e-commerce product catalog replicated across all regional servers.
 - **Partial Replication:** Only selected fragments are replicated.
 - *Example:* Frequently accessed customer data is replicated, but historical logs are not.

Advantages:

- Increased availability (fault tolerance).
- Faster read access (local copies).

Disadvantages:

- Higher storage cost.
- Update overhead (must sync all copies).

Full Replication (All Data Everywhere)

- The `Posts` table is copied to servers in:
 - North America
 - Europe
 - Asia

Behavior:

- A user in Tokyo sees the same posts as a user in New York.
- Writes must sync globally (higher update overhead).

Partial Replication (Hot Data Only)

- Only trending posts (`Posts_Trending`) are replicated.
- Older posts (`Posts_Archive`) are stored in a single location.

```
-- Trending posts (served from local replica)
SELECT * FROM Posts_Trending WHERE topic = 'WorldCup';

-- Archived posts (fetched from central server)
SELECT * FROM Posts_Archive WHERE year = 2010;
```

Comparison: Fragmentation vs Replication

Feature	Fragmentation	Replication
Storage Cost	Lower (no duplicates)	Higher (multiple copies)
Query Speed	Depends on joins	Faster reads (local access)
Update Cost	Lower (single copy)	Higher (sync all copies)
Fault Tolerance	Low (if a fragment is lost)	High (copies available)

Summary

- **Client/Server:** Centralized control, good for structured environments.
- **P2P:** Decentralized, resilient but harder to manage.
- **Multidatabase:** Integrates diverse DBs, useful for enterprises.
- **Fragmentation:** Improves performance by splitting data.
- **Replication:** Enhances availability by duplicating data.

Consistency Models (Strong, Eventual, Causal)

Consistency models define **how and when updates to a distributed database become visible to users**. Different models offer trade-offs between **performance, availability, and correctness**.

1. Strong Consistency

◆ Meaning

Every read returns the **most recent** write.

All users see the **same data immediately** after update.

◆ Example

- A bank account balance is updated from Rs. 5000 → Rs. 6000.
- Any user checking the balance **right after the update** will see Rs. 6000.
- No stale (old) data is ever shown.

◆ Where used?

- Banking systems
- Reservation systems
- File systems like HDFS (NameNode metadata)



2. Eventual Consistency

◆ Meaning

Data may be **temporarily inconsistent**, but **all replicas will become consistent eventually** if no new updates occur.

◆ Example

- You update your profile photo on Facebook.
- Some friends see the **new photo immediately**, others see the **old photo for a while**.
- After some time, all replicas sync, and everyone sees the **new photo**.

◆ Where used?

- Social networks
- DNS systems
- Distributed caches (e.g., Amazon DynamoDB)

✓ 3. Causal Consistency

◆ Meaning

If one operation **causes** another, all users must see them in the **same causal order**.

Independent operations can be seen in different orders.

◆ Example

1. User A posts: "I passed the exam!"
2. User B comments: "Congratulations!"

Causal consistency ensures:

- Everyone must see the post **before** they see the comment.

However, unrelated posts by others may appear in different orders.

◆ Where used?

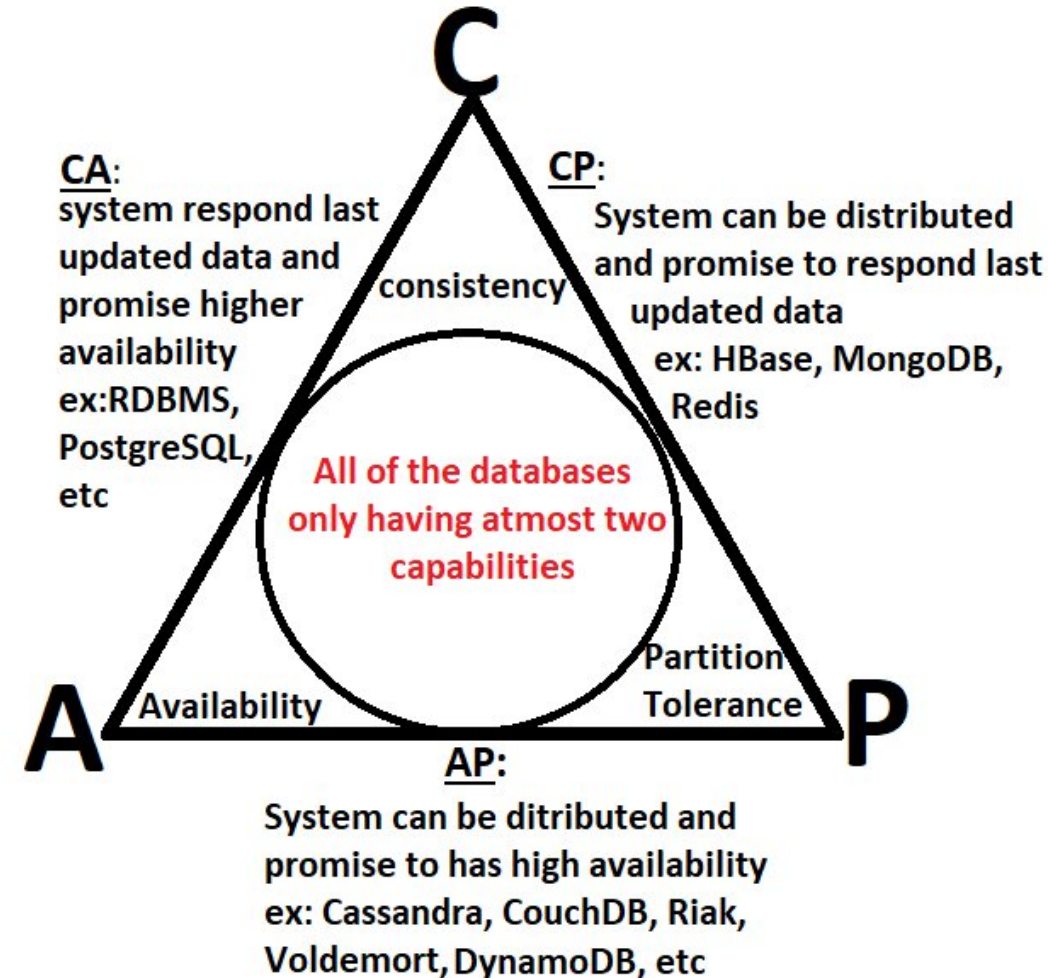
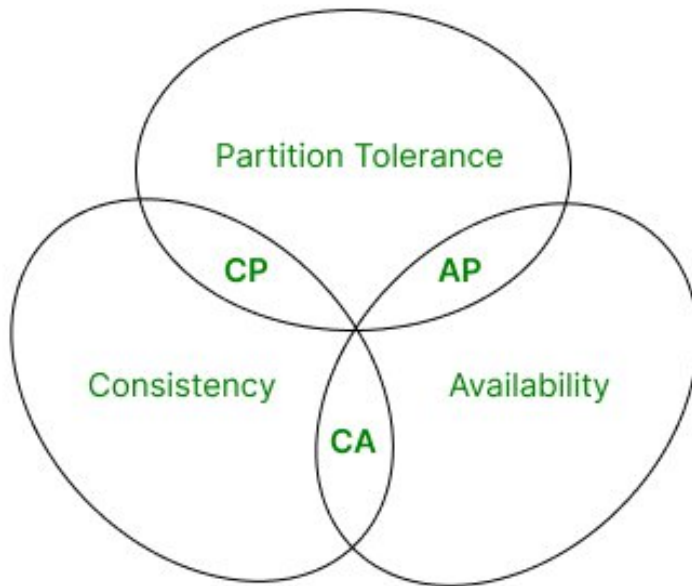
- Social media comments
- Collaborative applications
- Conversation threads

Consistency Model	What it Guarantees	Example
Strong	Always latest value	Bank balance update
Eventual	Becomes consistent after some time	Social media profile update
Causal	Maintains cause–effect order	Post → Comment

CAP Theorem & PACELC Trade-offs

1. CAP Theorem (Brewer's Theorem)

- **Definition:** In a distributed system, you can only guarantee **two out of three** properties:
 - **Consistency (C):** All nodes see the same data at the same time.
 - **Availability (A):** Every request receives a response (success/failure).
 - **Partition Tolerance (P):** The system continues working despite network failures.



Why Can't You Have All Three?

- In a **network partition (P)**, the system must choose:
 - **Cancel the request (CP)**: Ensures consistency but sacrifices availability.
 - **Proceed with stale data (AP)**: Ensures availability but sacrifices consistency.

Trade-offs in CAP Theorem

System Type	Guarantees	Sacrifices	Example
CA (Consistency + Availability)	Strong consistency, always available	No partition tolerance (single-node DB)	Traditional RDBMS (MySQL, PostgreSQL)
CP (Consistency + Partition Tolerance)	Data remains consistent, tolerates partitions	May become unavailable during partitions	MongoDB (configurable), HBase, ZooKeeper
AP (Availability + Partition Tolerance)	Always responds, tolerates partitions	May return stale data (eventual consistency)	Cassandra, DynamoDB, Riak

2. PACELC Extension of CAP

- **Definition:** Refines CAP by considering **latency (L)** and **consistency (C)** even when there is **no partition (E)**.
- **PACELC Rule:**
 - **If Partition (P) occurs:** Choose between **Availability (A)** or **Consistency (C)** (same as CAP).
 - **Else (E), under normal operation:** Choose between **Latency (L)** or **Consistency (C)**.

◆ What does Latency (L) mean in the PACELC Theorem?

Latency refers to the **time it takes for a system to respond** to a request — like how long it takes to read or write data.

In distributed databases, ensuring **strong consistency** often requires coordination between multiple nodes (e.g., waiting for acknowledgments), which increases latency.

On the other hand, if you relax consistency (e.g., allow reads from potentially outdated replicas), you can get faster responses — **lower latency**, but at the cost of possibly reading stale data.

PACELC Trade-offs

System Type	Partition Handling (P)	Normal Operation (E)	Example
PC/EC	Sacrifices Availability (A) for Consistency (C)	Sacrifices Latency (L) for Consistency (C)	Google Spanner, CockroachDB
PA/EL	Sacrifices Consistency (C) for Availability (A)	Sacrifices Consistency (C) for Low Latency (L)	Cassandra, DynamoDB
PC/EL	Sacrifices Availability (A) for Consistency (C)	Sacrifices Consistency (C) for Low Latency (L)	Rare (conflicting goals)

✔ Simple Example: Social Media App

Let's say you're using a social media app and update your profile picture.

- You upload a new photo.
- Your friend sees the new photo immediately → **Consistency**
- Your friend sees the old photo for a few seconds → **Lower latency, eventual consistency**

Scenario 1: High Consistency, Higher Latency

- The app waits until all servers around the world have the new photo before confirming the upload.
- ✓ Consistent everywhere
- ✗ Slower response time

Scenario 2: Lower Latency, Eventual Consistency

- The app confirms the upload as soon as one server has the new photo.
- Other servers will get the update shortly.
- ✓ Fast response
- ✗ Temporarily inconsistent data

This trade-off between latency and consistency is exactly what the "E" part of PACELC is about.

So in **PACELC** , even when everything is working fine (**no network issues**), you still have to decide:

👉 Do you want **faster responses** (low latency), or do you want **always-updated data** (strong consistency)?

Why PACELC Matters?

- CAP only considers **failure scenarios (partitions)**, but PACELC also considers **performance trade-offs in normal operations**.
- Example:
 - A **banking system (PC/EC)** ensures strong consistency but may have higher latency.
 - A **social media feed (PA/EL)** prioritizes fast reads (low latency) but may show slightly stale data.

Concept	Key Idea	Use Case
CAP Theorem	Choose 2 out of 3 (C, A, P)	Helps pick DBs for fault tolerance vs. consistency
PACELC	Extends CAP by adding Latency vs. Consistency trade-offs	Helps optimize performance in normal & failure cases

Real-World Examples

- **CP Systems (Banking, Financial Transactions):** Need strict consistency.
- **AP Systems (Social Media, IoT Sensors):** Prioritize availability over immediate consistency.
- **CA Systems (Single-Node Databases):** Not distributed (no partition tolerance).

What are the advantages and potential drawbacks of vertical fragmentation in distributed databases? Given the set of attributes and their access frequencies, describe the process to create vertical fragments using clustered affinity matrix. Assumptions can be made if necessary. [PU, 2025]

Advantages of Vertical Fragmentation

1. Improved Performance

Each site stores only the required attributes (columns), reducing the amount of data accessed and transferred during queries.

2. Better Locality of Reference

Applications at each site can access only the attributes they frequently use, improving response time.

3. Enhanced Security

Sensitive attributes (e.g., salary) can be stored separately, allowing better control and protection.

4. Reduced Storage and Communication Cost

Only relevant attributes are stored and transmitted, lowering overhead.

Potential Drawbacks of Vertical Fragmentation

1. Costly Reconstruction (Join Overhead)

To answer queries needing attributes from multiple fragments, the system must perform joins across sites, increasing delay.

2. Complexity in Query Processing

Query optimization becomes more difficult because the DBMS must track where each attribute is stored.

3. Difficult Consistency Management

Updates across multiple fragments (sites) must be synchronized, increasing coordination overhead.

4. Possible Data Redundancy

If primary keys must be repeated in each fragment, it increases storage and update cost.

Affinity Matrix (Attribute Affinity Matrix)

An **affinity matrix** is a square table used in distributed database design to show **how closely related attributes are**, based on how often they are accessed together in queries.

- Rows and columns represent the **attributes** of a relation.
- Each cell **Affinity(A_i, A_j)** shows the **total access frequency** of queries that use **both attributes A_i and A_j together**.

Let's design vertical fragments for a Student table with attributes:

Roll_No (PK), Name, Dept, GPA, Hostel, Fees

Queries & Frequencies:

Q1: Get Name, Dept, GPA (Admissions office) → **40 times/day**

Q2: Get Roll_No, Hostel, Fees (Finance office) → **30 times/day**

Q3: Get Roll_No, Name, Dept (ID card system) → **20 times/day**

Q4: Get Roll_No, GPA (Scholarship check) → **10 times/day**

Step 1: Attribute Usage Matrix

Attribute	Q1(40)	Q2(30)	Q3(20)	Q4(10)
Roll_No	0	1	1	1
Name	1	0	1	0
Dept	1	0	1	0
GPA	1	0	0	1
Hostel	0	1	0	0
Fees	0	1	0	0

Step 2: Build Affinity Matrix

For each attribute pair:

aff(X,Y) = sum of frequencies of queries accessing BOTH

Let's calculate a few:

Complete Affinity Matrix:

	R	N	D	G	H	F
Roll_No	-	20	20	10	30	30
Name	20	-	60	40	0	0
Dept	20	60	-	40	0	0
GPA	10	40	40	-	0	0
Hostel	30	0	0	0	-	30
Fees	30	0	0	0	30	-

1. Roll_No & Name:

Q3 accesses both (20)
Q1 has Name but not Roll_No
Q2 has Roll_No but not Name
Q4 has Roll_No but not Name
aff = 20

2. Roll_No & GPA:

Q4 accesses both (10)
aff = 10

3. Name & Dept:

Q1 accesses both (40)
Q3 accesses both (20)
aff = 60

4. Hostel & Fees:

Q2 accesses both (30)
aff = 30

Step 3: Cluster by Inspection

Look for **high values** (closely used together):

1. **Name–Dept–GPA** group:

Name–Dept = 60

Name–GPA = 40

Dept–GPA = 40

All three often accessed together (Q1)

2. **Hostel–Fees** group:

Hostel–Fees = 30

Both accessed only by Q2

3. **Roll_No** connects to both groups:

- Roll_No with Hostel/Fees = 30 (strong)
- Roll_No with Name/Dept = 20 (medium)
- Roll_No with GPA = 10 (weak)

Step 4: Logical Grouping

Group A (Academic Info): Name, Dept, GPA

Group B (Administrative): Hostel, Fees

Roll_No (PK) must be in **both fragments** to join them.

So fragments:

Fragment 1: Roll_No, Name, Dept, GPA

Fragment 2: Roll_No, Hostel, Fees

Step 5: Check Efficiency

Transaction analysis:

- Q1: Accesses Fragment 1 only (40/day) ✓
- Q2: Accesses Fragment 2 only (30/day) ✓
- Q3: Accesses Fragment 1 only (20/day) ✓
- Q4: Accesses Fragment 1 only (10/day) ✓

No transaction accesses both fragments in this example! Perfect separation.

Distributed Transactions

Distributed transactions ensure **atomicity and consistency** across multiple nodes. Key protocols include:

1) Two-Phase Commit (2PC)

Definition:

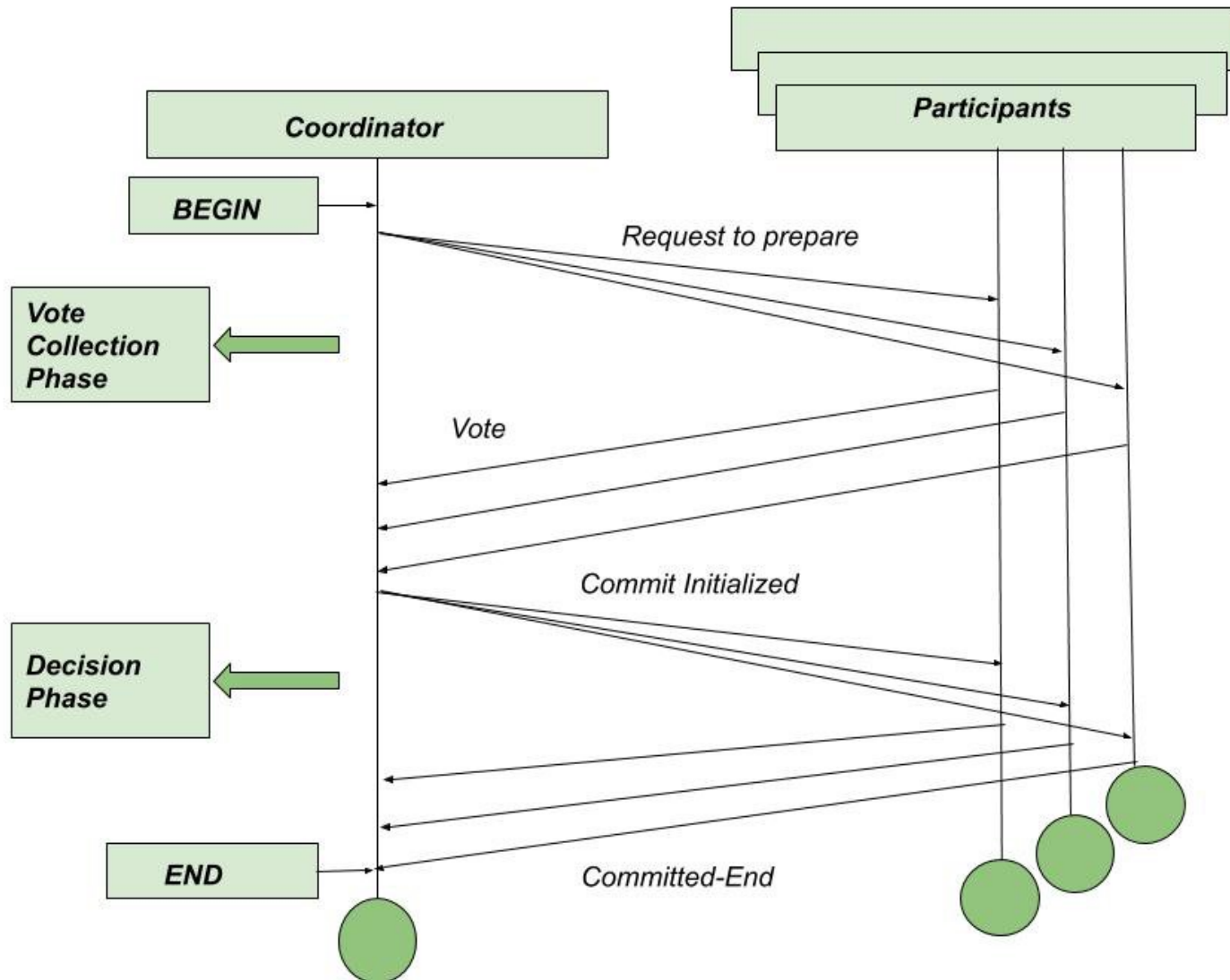
2PC is a distributed transaction protocol that ensures **atomicity** (all-or-nothing execution) across multiple nodes (participants).

Key Characteristics:

- Guarantees **strong consistency**.
- **Blocking protocol**: if the coordinator fails after participants vote YES, they may block waiting.

Where Useful:

- Banking systems (money transfer across branches)
- Distributed databases needing strict consistency
- Reservation systems (seat/room booking)



Phases:

1. Prepare (Voting) Phase:

1. Coordinator asks all participants: "Can you commit?"
2. Each participant replies **YES (ready)** or **NO (abort)**.

2. Commit Phase:

1. If all vote YES → coordinator sends **COMMIT**.
2. If any vote NO → coordinator sends **ABORT**.

2) Three-Phase Commit (3PC)

Definition:

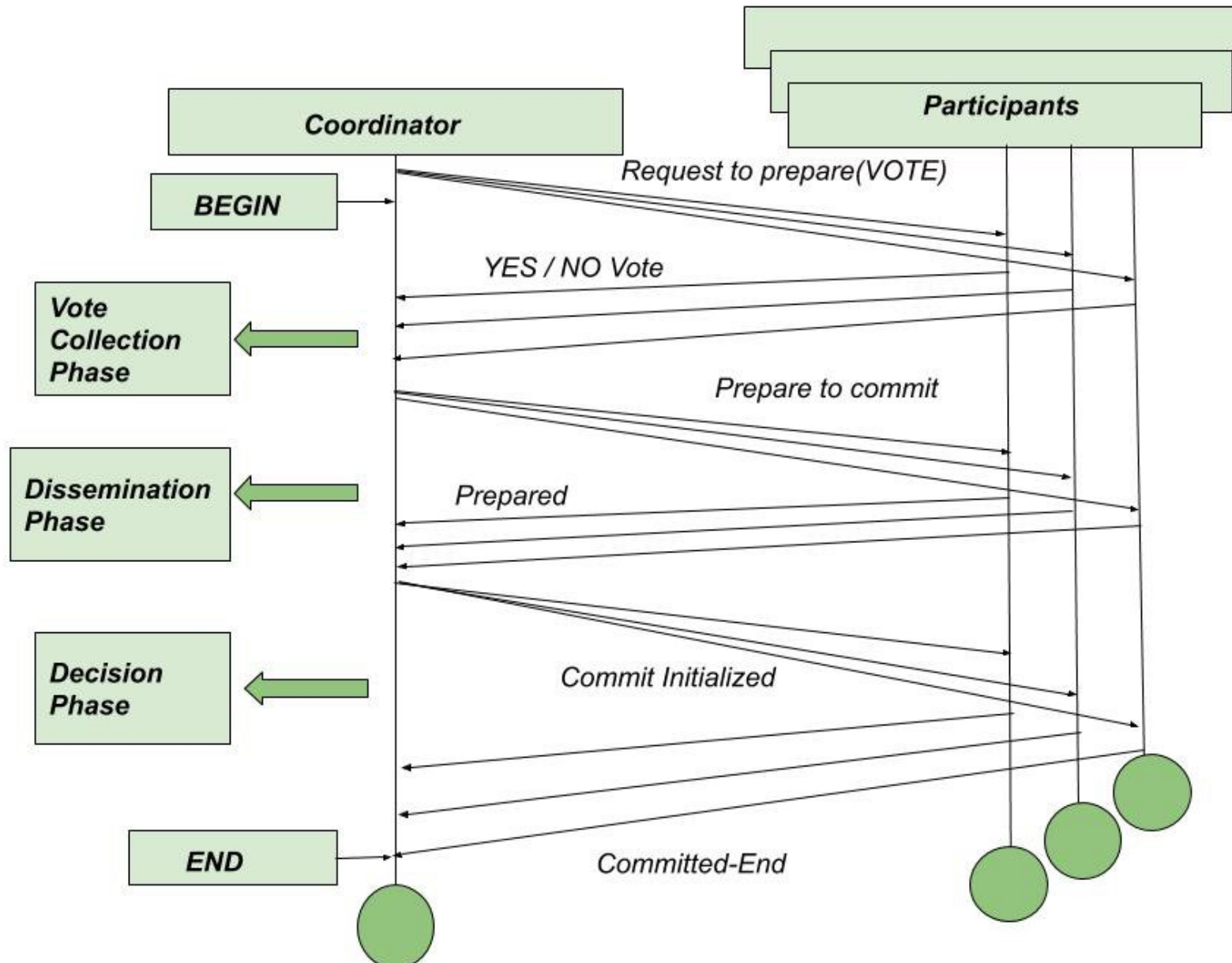
3PC improves 2PC by reducing blocking, adding an extra phase to handle failures better.

Key Characteristics:

- **Non-blocking** (under certain assumptions).
- Requires **timeout mechanisms** and assumes no network partition.
- More messages → more overhead than 2PC.

Where Useful:

- Distributed systems where **availability is critical**
- Systems tolerating extra overhead to reduce blocking
- Real-time or fault-sensitive environments



Phases:

1.CanCommit?

1. Coordinator checks if participants can commit.

2.PreCommit:

1. Participants acknowledge readiness and enter a safe state.

3.DoCommit:

1. Coordinator sends final commit instruction.

Examples: 2PL & 3PL

1. Two-Phase Commit (2PC)

Purpose: Ensures atomicity (all nodes commit or all abort) in distributed transactions.

Example: Bank Transfer Across Branches

- **Coordinator:** Main bank server
- **Participants:** Branch A (Sender) & Branch B (Receiver)

Steps:

1. Prepare Phase:

- Coordinator asks: *"Can Branch A debit \$100 and Branch B credit \$100?"*
- Both branches lock their data and reply **"Yes"** if ready.

2. Commit Phase:

- If **all** say **"Yes"**, coordinator sends **"Commit"**.
- If **any** says **"No"**, coordinator sends **"Abort"**.

✓ **Pros:** Ensures no partial updates (atomicity).

✗ **Cons: Blocking**—if coordinator crashes, branches wait forever.

2. Three-Phase Commit (3PC)

Purpose: Makes 2PC **non-blocking** by adding a **pre-commit** phase.

Same Bank Transfer Example

1. CanCommit? (Voting)

- Branches vote **"Yes"** if ready.

2. PreCommit (Prepare to commit)

- Coordinator confirms: *"Everyone agreed, get ready!"*

3. DoCommit (Finalize)

- If all acknowledge pre-commit, money moves.

✓ **Pros:** Survives coordinator failure (branches can timeout and abort).

✗ **Cons:** More messages = **slower** than 2PC.

Paxos and Raft (Consensus Algorithm in Distributed System)

In a **distributed system**, data and services are spread across multiple computers (nodes).

Because of **network delays, crashes, or failures**, different nodes may have **different views of data**.

So, a key problem is:

How do all nodes agree on the same decision or data value?

This problem is called **distributed consensus**.

Main Purpose of Paxos and Raft

The main purpose of Paxos and Raft is to achieve consensus — that is, to make **all nodes agree on one value or order of operations**, even if some nodes fail.

They help maintain:

- Consistency
- Fault tolerance
- Reliable replication

Paxos

Paxos is a method used in **distributed systems** so that **all computers agree on one decision**, even if some computers fail.

In simple words:

👉 *Many computers discuss and finally agree on the same value.*

Simple example:

Several database servers must agree **whether to save an update or not**. Paxos helps them reach the same decision.

Raft

Raft is also a method to make **all computers agree**, but it is **easier to understand** than Paxos.

In simple words:

👉 *One computer becomes the leader and others follow its decision.*

Simple example:

In a server cluster, one **leader server** decides the order of client requests and copies it to other servers.

Paxos – Real Scenario

Scenario: Distributed Database Replication

A company has a **banking database** replicated on **3 servers** in different locations.

When a transaction (₹10,000 transfer) occurs, **all replicas must agree** whether to commit it.

How Paxos helps:

- Servers exchange proposals.
- Even if **one server crashes**, the remaining servers **agree on the same decision**.
- Ensures **data consistency**.

Where Paxos is used:

- Google Chubby lock service
- Some distributed databases
- Theoretical foundation for consensus

Raft – Real Scenario

Scenario: Cloud Server Cluster

A cloud system has 5 servers running a key-value store.

- One server is elected as **leader**.
- Clients send requests only to the leader.
- Leader replicates updates to followers.

How Raft helps:

- If the leader fails, a **new leader is elected automatically**.
- All servers keep the **same log of updates**.

Where Raft is mostly used (very important):

- etcd (Kubernetes)
- Consul
- Apache Kafka (KRaft mode)
- Distributed configuration systems

Which is Mostly Used?

👉 **Raft is more widely used in practice** because:

- Easier to understand and implement
- Clear leader-based design
- Better for real-world systems

👉 **Paxos is:**

- Powerful but complex
- More common in research and foundational systems

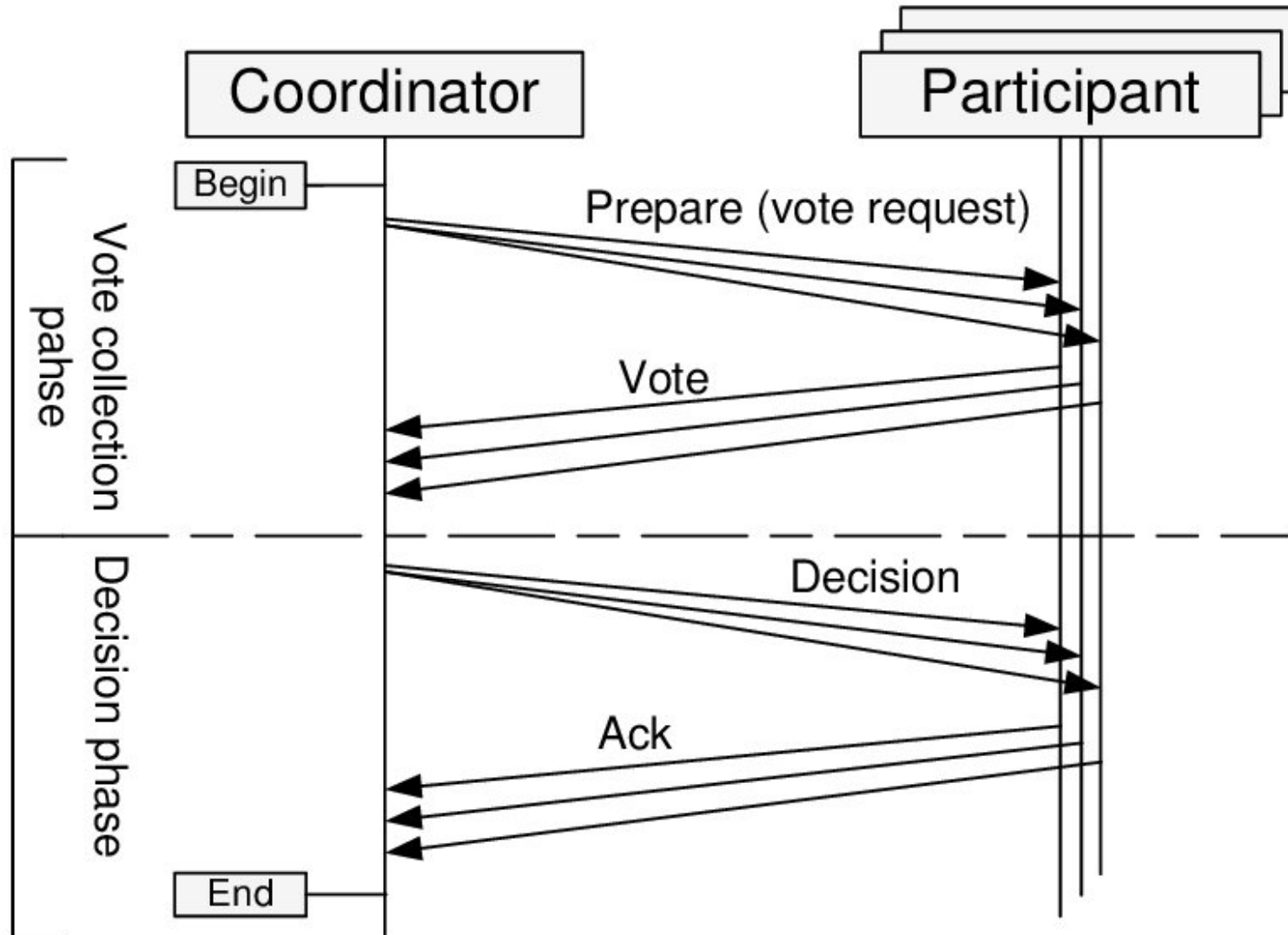
How does the two phase commit 2PC protocol differ from the Three Phase Commit 3PC Protocol? Provide a detailed comparison including their communication structure and state transition diagram.

1. Overview

Feature	2-Phase Commit (2PC)	3-Phase Commit (3PC)
Purpose	Ensure atomic commit in distributed transactions	Solve blocking problem in 2PC and make it non-blocking
Blocking?	Yes (blocking protocol)	Non-blocking
Failure handling	Limited, coordinator failure blocks participants	Better timeout handling, participants can make decisions
Number of phases	2	3
Safe commit?	Only after all votes	Introduces an intermediate “Pre-commit” to ensure safety

✓ 2. Phases and Communication Structure

A. Two-Phase Commit (2PC)



Phase 1: Voting Phase

- **Coordinator → Participants:** *“Prepare / Can you commit?”*
- **Participants → Coordinator:** *“Vote-Commit” or “Vote-Abort”*

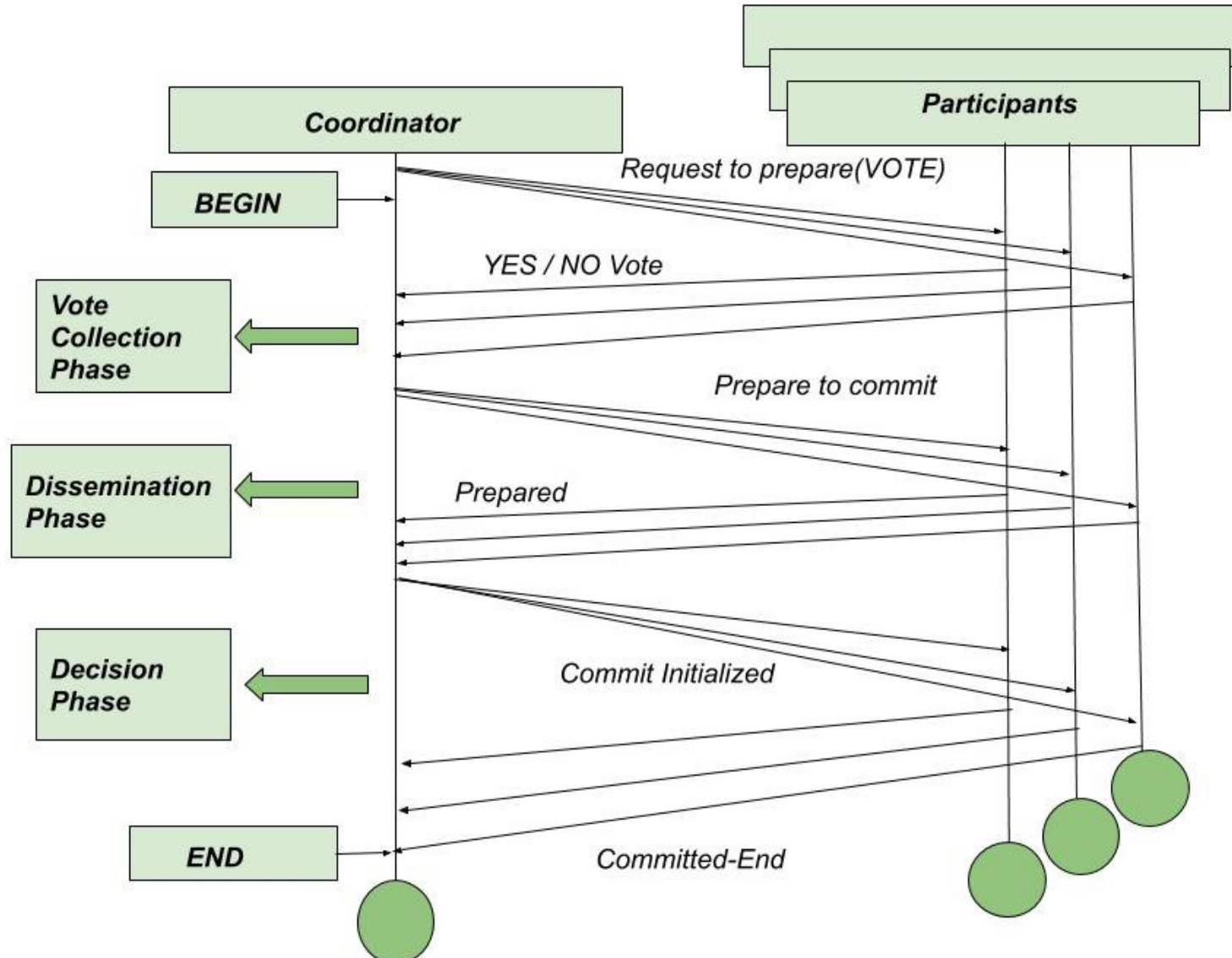
Phase 2: Decision Phase

- If all vote Commit → **Coordinator broadcasts “Global Commit”**
- If any vote Abort → **Coordinator broadcasts “Global Abort”**

Major Issue:

If the coordinator crashes after sending prepare but before sending decision → participants remain blocked.

B. Three-Phase Commit (3PC)



Phase 1: CanCommit? Phase

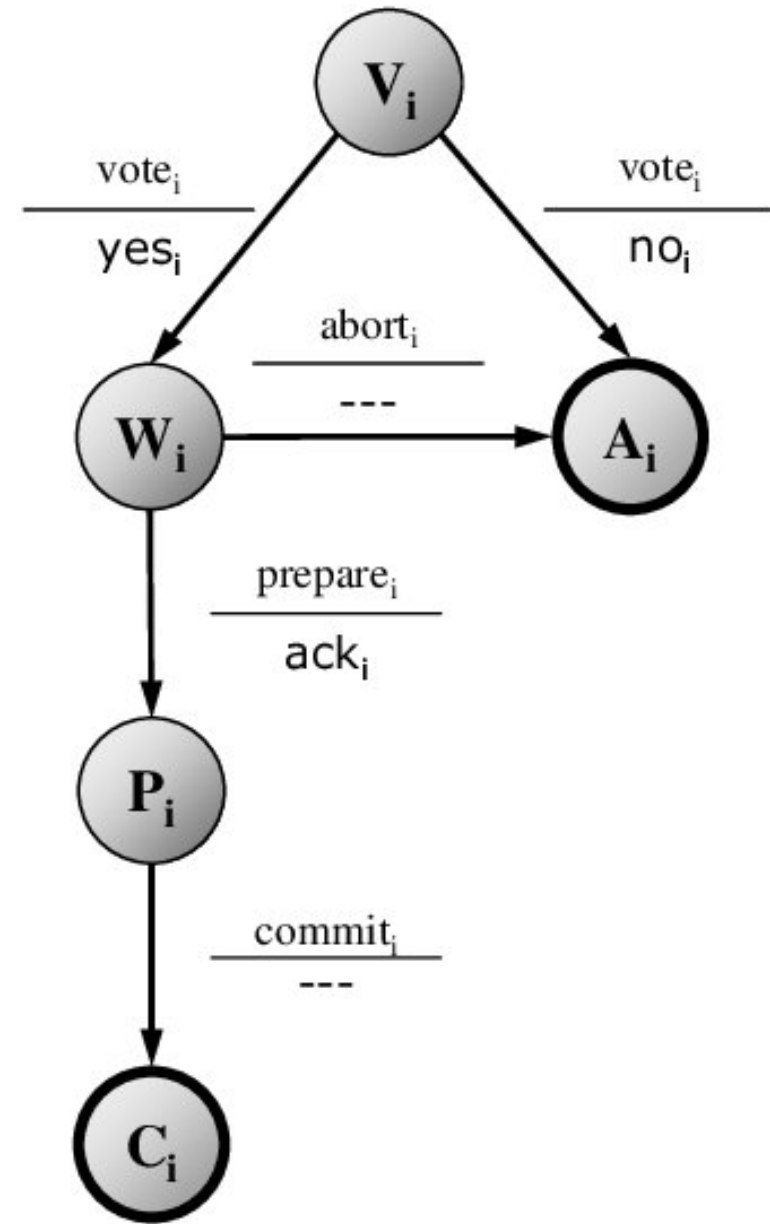
Same as 2PC voting phase:

- Coordinator asks: “*Can you commit?*”
- Participants vote: *Yes / No*

Phase 2: PreCommit Phase

If all participants vote Yes:

1. Coordinator → Participants: “*Pre-Commit (prepare to commit)*”
2. Participants enter Prepared/Pre-Commit state
3. Start timeout → if coordinator fails, they may commit safely later.



V_i	Vote
W_i	Wait
A_i	Abort
P_i	Prepare
C_i	Commit

Phase 3: DoCommit Phase

1. Coordinator $\rightarrow$ Participants: “*Do Commit*”
2. Participants commit and complete transaction.

Key Advantage

PreCommit ensures all participants are consistent, so even if coordinator crashes, they can safely decide using timeout rules.

Comparisons between 2PC and 3PC

1. Number of Phases

- 2PC: 2 (Prepare, Commit)
- 3PC: 3 (CanCommit, PreCommit, DoCommit)

2. Blocking vs Non-Blocking

Protocol	Blocking?	Why?
2PC	YES	Participants may wait in READY forever if coordinator fails
3PC	NO	Timeout + PreCommit allow autonomous decisions

3. Coordinator Failure Handling

- 2PC:
 - If failure after "prepare", participants wait forever (uncertain state).
- 3PC:
 - Timeout + Pre-commit state allow safe commit.

4. Participant Failure Handling

- **2PC:**
 - Coordinator blocks transaction until participant recovers.
- **3PC:**
 - Coordinator may proceed using majority/timeouts.

5. Network Partition Tolerance

- **2PC:** Poor tolerance (blocks)
- **3PC:** Better (can move to commit/abort based on state)

6. Message Complexity

- **2PC:** Requires fewer messages
- **3PC:** More messages (extra Pre-Commit round)

Feature	2PC	3PC
Blocking	Yes	No
Timeout-based decisions	No	Yes
Tolerates coordinator crash	Poor	Good
Phases	2	3
Pre-Commit stage	Absent	Present
Message Cost	Lower	Higher
Atomicity Guarantee	Strong	Strong + Non-blocking

Security in Distributed Databases

Security in a distributed database refers to the protection of data stored across multiple networked sites from unauthorized access, misuse, modification, or destruction, while ensuring safe and reliable data sharing among authorized users.

Main Aspects of Security

1. Authentication (Who are you?)

Verifies the identity of a user or system.

Example:

A user must log in with a username and password or OTP before accessing a distributed university database.

2. Authorization (What can you access?)

Controls what operations a user is allowed to perform.

Example:

A **student** can view marks, but only the **admin** can modify grades across all database sites.

3. Confidentiality (Data Privacy)

Ensures data is not disclosed to unauthorized users.

Example:

Employee salary data is **encrypted** before being transmitted between branch databases.

4. Integrity (Data Correctness)

Ensures data is not altered improperly during transmission or storage.

Example:

Hashing and checksums ensure that data sent between distributed sites is **not modified**.

5. Availability (Access When Needed)

Ensures data is available even during failures or attacks.

Example:

Database replication allows users to access data from another site if one server fails.

How Security is Achieved in Distributed Databases

1. Authentication (Who are you?)

How it is achieved:

- Username & password
- OTP / multi-factor authentication
- Digital certificates

In DDBMS:

User identity is verified **before accessing any database site.**

2. Authorization (What can you access?)

How it is achieved:

- Roles and privileges (GRANT / REVOKE)
- Role-Based Access Control (RBAC)

In DDBMS:

Each site enforces access rules so users can only **read or modify permitted data**.

4. Integrity (Data Correctness)

How it is achieved:

- Hashing and checksums
- Transaction management (ACID properties)
- Commit protocols (2PC)

In DDBMS:

Ensures data is **not altered incorrectly** during transmission or updates.

3. Confidentiality (Data Privacy)

How it is achieved:

- Data encryption (AES, SSL/TLS)
- Secure communication channels

In DDBMS:

Data transferred between sites is **encrypted** to prevent eavesdropping.

5. Availability (Access When Needed)

How it is achieved:

- Data replication
- Fault tolerance
- Backup and recovery

In DDBMS:

If one site fails, another site **continues serving data**.

Security Aspect	How Achieved
Authentication	Login, OTP, certificates
Authorization	Roles, privileges
Confidentiality	Encryption
Integrity	Hashing, transactions
Availability	Replication, backups

Case Study / Scenario: Distributed University Database System

A university has a **distributed database** spread across:

- Central Campus Server
- Examination Office Server
- Department Servers

The database stores **student records, marks, salaries, and attendance.**

Security Applications in the Scenario

1. Authentication

A teacher must **log in** with a **username** and **password** to access the examination database.

2. Authorization

- Teachers can **enter marks** for their own subjects only.
- Students can **view marks** but **cannot modify** them.
- Admin can access data at **all sites**.

3. Confidentiality

Student results and staff salaries are **encrypted** while being transferred between campus servers.

4. Integrity

Checksums ensure that marks sent from department servers to the central server are **not altered**.

5. Availability

If the central server fails, data is accessed from a **replica server** so students can still view results.

Security in distributed databases protects data across multiple sites by ensuring authentication, authorization, confidentiality, integrity, and availability.

Homework [SOE, pre-board]

7. Explain the difference between strong consistency, eventual consistency and casual consistency with suitable examples.
8. Can a System be strongly Consistent and highly available at the same time? Relate your answers to the CAP Theorem.